Università degli Studi di Napoli – Federico II

Controllo cablato e microprogrammato

ARCHITETTURA *dei* **SISTEMI DIGITALI**

Prof.ssa Alessandra De Benedictis

CONTROLLO CABLATO VS MICROPROGRAMMATO

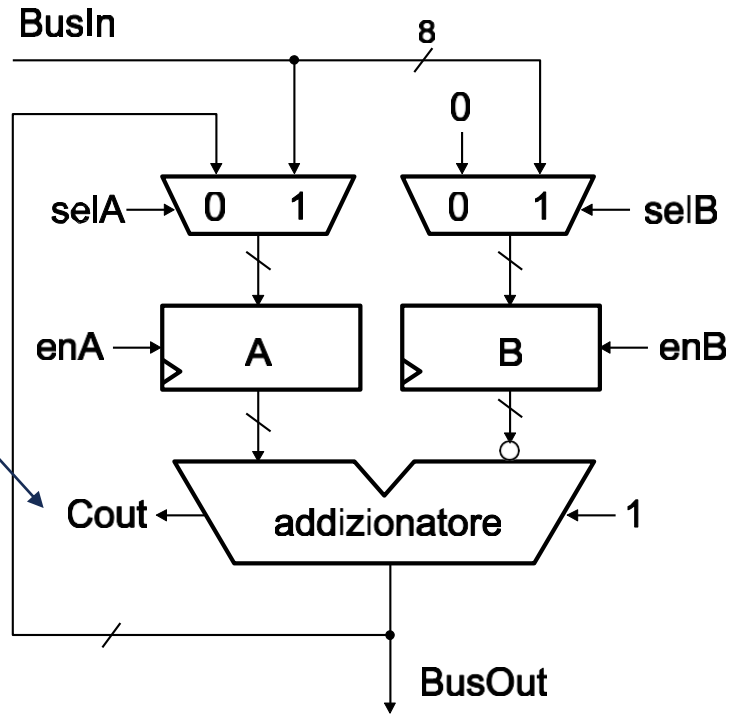
- Le FSM rappresentano il modo più naturale per implementare la control unit in un sistema digitale: gli stati della FSM rappresentano i diversi stati del controllore, le uscite della FSM in ogni stato rappresentano i segnal di controllo forniti al data path (in tal caso si parla di **controllo cablato**)

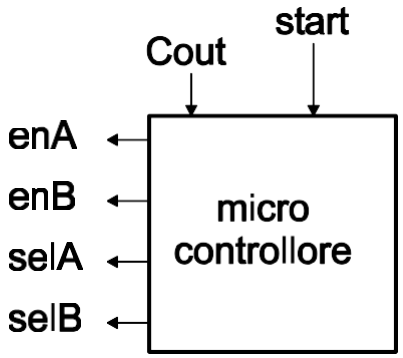
- Un modo alternativo per realizzare il controllo del data path di una architettura si basa sul ricorso ad una memoria ROM contenente un vero e proprio "microprogramma".
  - Ogni parola contenuta nella memoria (microistruzione) fornisce (eventualmente in combinazione con logica esterna) i segnali di controllo da attivare in un singolo «passo» di elaborazione.
  - Un registro micro Program Counter (microPC) indirizza la memoria individuando la microistruzione da attivare in ogni istante
    - Per l'aggiornamento del microPC è possibile prevedere che ogni parola di controllo contenga essa stessa un gruppo di bit che forniscono il valore successivo. Tale valore può essere eventualmente modificato mediante opportuna logica combinatoria esterna per consentire di realizzare salti

ESEMPIO: A MOD B

- Consideriamo un sistema che legge due numeri A e B di 8 bit e calcola A mod B
- Il data path dell'architettura è composto da due registri, due mux e un sottrattore

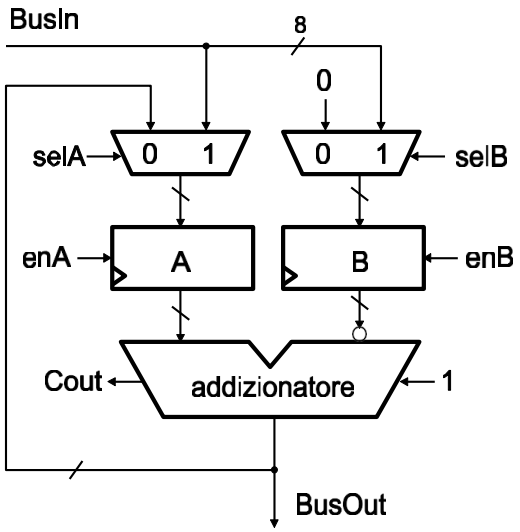
A e B sono numeri positivi su n bit e il sommatore è a n bit. Il riporto Cout rappresenta il Quando Cout=1 il ris è positivo, altrimenti è negativo

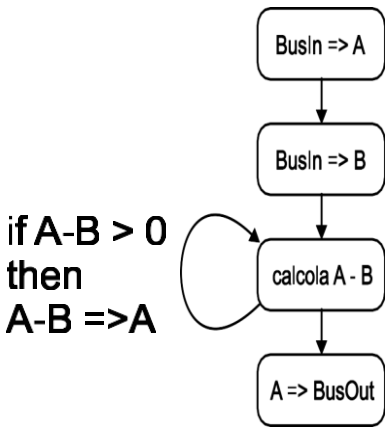




ESEMPIO: A MOD B – CONTROLLO CABLATO

- Il controllore del nostro sistema dovrà presentare una serie di stati in cui dovranno essere settati gli opportuni segnali perché vengano effettuate le operazioni corrispondenti:





- (in1) Si carica A
- (in2) Si carica B
- (op) Si calcola A-B. Se il risultato è positivo, esso viene posto nel registro A e si ripete il passo 2; Se il risultato è negativo, A non viene sovrascritto e invece si carica 0 in B
- (end) Il risultato del sommatore (che corrisponde con il contenuto di A prima della sottrazione che ha dato risultato negativo) viene presentato sul bus di uscita BusOut

in1: selA =1 enA =1 enB =0

in2: selB =1 enB =1 enA =0

op: *if* Cout1=1 *then*
selA =0
enA =1
goto **op**
else
enA =0
selB = 0
enB=1

end: nop

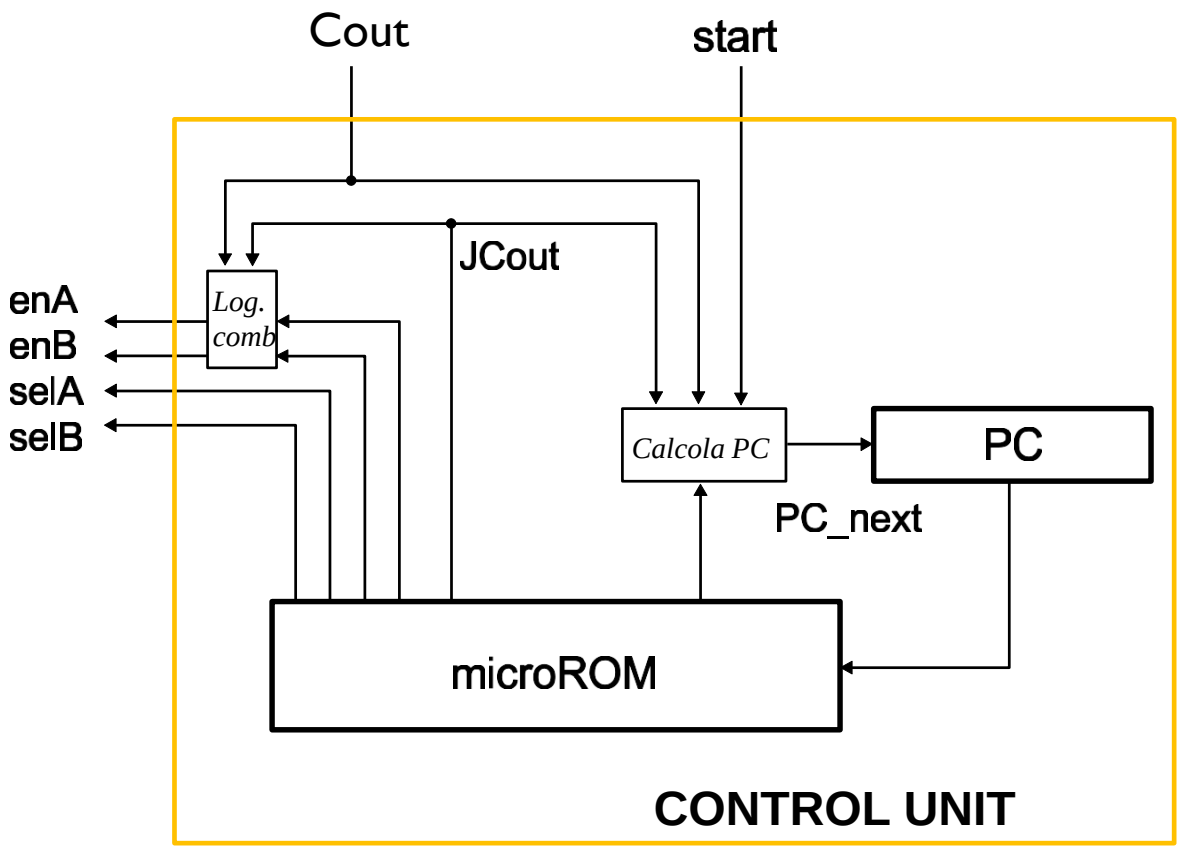
ESEMPIO: A MOD B – CONTROLLO CABLATO - VHDL

```
LIBRARY IEEE ;
USE IEEE.std_logic_1164.all;
USE work.all;

ENTITY controller IS
          PORT (
          clk                 :          IN std_logic;
          start               :          IN std_logic;
          Cout                :          IN std_logic;
          selA                :          OUT std_logic;
          enA                 :          OUT std_logic;
          selB                :          OUT std_logic;
          enB                 :          OUT std_logic    );
END controller;

ARCHITECTURE two_process of controller IS
          TYPE state IS (
          idle,
          in1,
          in2,
          op                  );

          SIGNAL current_state, next_state : state;

BEGIN
```

```
reg_state: PROCESS (clk)
BEGIN
IF (clk'event and clk = '1') THEN
          IF (start = '1') THEN     current_state <= in1;
          ELSE      current_state <= next_state;
          END IF;
END IF;
END PROCESS reg_state;

Comb: PROCESS (current_state, Cout) IS
BEGIN
          selA <= '0'; enA <= '0';      -- valori asegnati per default
          selB <= '0'; enB <= '0';      --
          next_state <= idle;           --
          CASE current_state IS
          WHEN idle =>        next_state <= idle;
          WHEN in1 =>         selA                <= '1';
                              enA                 <= '1';
                              next_state          <= in2;
          WHEN in2 =>         selB                <= '1';
                              enB                 <= '1';
                              next_state          <= op;
          WHEN op =>          IF Cout='1' THEN
                                        enA                 <= '1';
                                        next_state          <= op;
                              ELSE      enB                 <= '1';
                                        next_state          <= idle;
                              END IF;
          END case;
END PROCESS;
END two_process;
```

Nota: in op in base al valore di Cout si modifica il flusso di controllo

ESEMPIO: A MOD B – CONTROLLO MICROPROGRAMMATO



- L'unità di controllo si serve di una microROM in cui sono contenute le parole corrispondenti alle «microistruzioni» da eseguire.
- In ogni istante la ROM prende in ingresso il PC corrente e fornisce in uscita i bit della parola letta.
- Le microistruzioni corrispondono agli stati della FSM del controllo cablato. Ogni microistruzione deve contenere:
  - i segnali da settare verso l'esterno (enA, selA, enB, selB)
  - l'indirizzo della prossima microistruzione (PC_next)
- Per modificare il flusso di controllo c'è bisogno di un meccanismo esterno alla ROM che consenta di caricare una microistruzione diversa da quella puntata da PC_next:
  - utilizziamo un segnale interno al controllore (JCout) che viene associato ad ogni microistruzione; JCout=1 quando sto eseguendo il passo «op» e so che devo guardare Cout per capire cosa fare (caricare in A il risultato della sottrazione oppure uscire)
  - Sulla base di JCout e Cout posso settare enA e enB e posso calcolare il nuovo PC

ESEMPIO: A MOD B – CONTROLLO MICROPROGRAMMATO - VHDL

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.all;
USE IEEE.std_logic_arith.all;
USE work.all;

ENTITY MicroROM IS
        PORT (    PC                  : IN unsigned(1 downto 0);
                  PC_next             : OUT unsigned(1 downto 0);
                  JCout               : OUT std_logic;
                  enA                 : OUT std_logic;
                  selA                : OUT std_logic;
                  enB                 : OUT std_logic;
                  selB                : OUT std_logic   );
END MicroROM;

ARCHITECTURE synth of MicroROM IS
```

```
TYPE Controllo_type IS RECORD
          PC_next             : unsigned(1 downto 0);
          JCout               : std_logic;
          enA                 : std_logic;
          selA                : std_logic;
          enB                 : std_logic;
          selB                : std_logic;
END RECORD;
```

Controllo_type definisce la struttura delle microistruzioni

```
CONSTANT idle : Controllo_type := (
          PC_next   => "00",
          JCout     => '0',
          enA       => '0',
          selA      => '0',
          enB       => '0',
          selB      => '0     );
CONSTANT in1 : Controllo_type := (
          PC_next   => "10",
          JCout     => '0',
          enA       => '1',
          selA      => '1',
          enB       => '0',
          selB      => '0     );
CONSTANT in2 : Controllo_type := (
          PC_next   => "11",
          JCout     => '0',
          enA       => '0',
          selA      => '0',
          enB       => '1',
          selB      => '1'    );
CONSTANT op : Controllo_type := (
          PC_next   => "11",
          JCout     => '1',
          enA       => '1',
          selA      => '0',
          enB       => '1',
          selB      => '0'    );
```

```
TYPE ROM_TYPE IS ARRAY(0 to 3) OF Controllo_type;

CONSTANT ROM : ROM_type := (
          0 => idle,
          1 => in1,
          2 => in2,
          3 => op
);

SIGNAL Controllo : Controllo_type;

BEGIN

Controllo <= ROM(conv_integer(PC));

          PC_next                    <= Controllo.PC_next;
          JCout                      <= Controllo.JCout;
          enA                        <= Controllo.enA;
          selA                       <= Controllo.selA;
          enB                        <= Controllo.enB;
          selB                       <= Controllo.selB;

END synth;
```

La ROM contiene 4 microistruzioni che corrispondono ai 4 «stati» del controllore: per come sono definite le microistruzioni, la loro sequenza è fissa, quindi da «op» dovrei continuare a caricare «op» per sempre

ESEMPIO: A MOD B – CONTROLLO MICROPROGRAMMATO - VHDL

```
LIBRARY IEEE ;
USE IEEE.std_logic_1164.all;
USE IEEE.std_logic_arith.all;
USE work.all;

ENTITY controller IS
          PORT (
          clk                 :          IN std_logic;
          start               :          IN std_logic;
          Cout                :          IN std_logic;
          selA                :          OUT std_logic;
          enA                 :          OUT std_logic;
          selB                :          OUT std_logic;
          enB                 :          OUT std_logic    );
END controller;

ARCHITECTURE microrom of controller IS

COMPONENT MicroROM IS
          PORT (
                    PC                    : IN unsigned(1 downto 0);
                    PC_next               : OUT unsigned(1 downto 0);
                    JCout                 : OUT std_logic;
                    enA                   : OUT std_logic;
                    selA                  : OUT std_logic;
                    enB                   : OUT std_logic;
                    selB                  : OUT std_logic      );
END COMPONENT;
```

```
SIGNAL PC_next, PC                    : unsigned(1 downto 0);
SIGNAL enA_tmp, enB_tmp, JCout        : std_logic;
BEGIN
          rom: MicroROM
          PORT MAP (
                    PC                  => PC,
                    PC_next             => PC_next,
                    JCout               => JCout,
                    enA                 => enA_tmp,
                    selA                => selA,
                    enB                 => enB_tmp,
                    selB                => selB    );

          reg_PC: PROCESS (clk)
          BEGIN
          IF (clk'event and clk = '1') THEN
                    IF     (start = '1') THEN       PC <= "01";
                    ELSIF (JCout='0')    THEN       PC <= PC_next
                    ELSE
                              IF (Cout='0') THEN
                                        PC <= "00";
                                        END IF;
                    END IF;
          END IF;
          END PROCESS reg_PC;

          enA <= enA_tmp       WHEN JCout='0' ELSE Cout;
          enB <= enB_tmp       WHEN JCout='0' ELSE not(Cout);

END microrom;
```

Questo process modifica il flusso di controllo in base a start e JCout: quando JCout=1 (cioè quando si esegue la microistruzione «op») si controlla cout (segno) per modificare PC

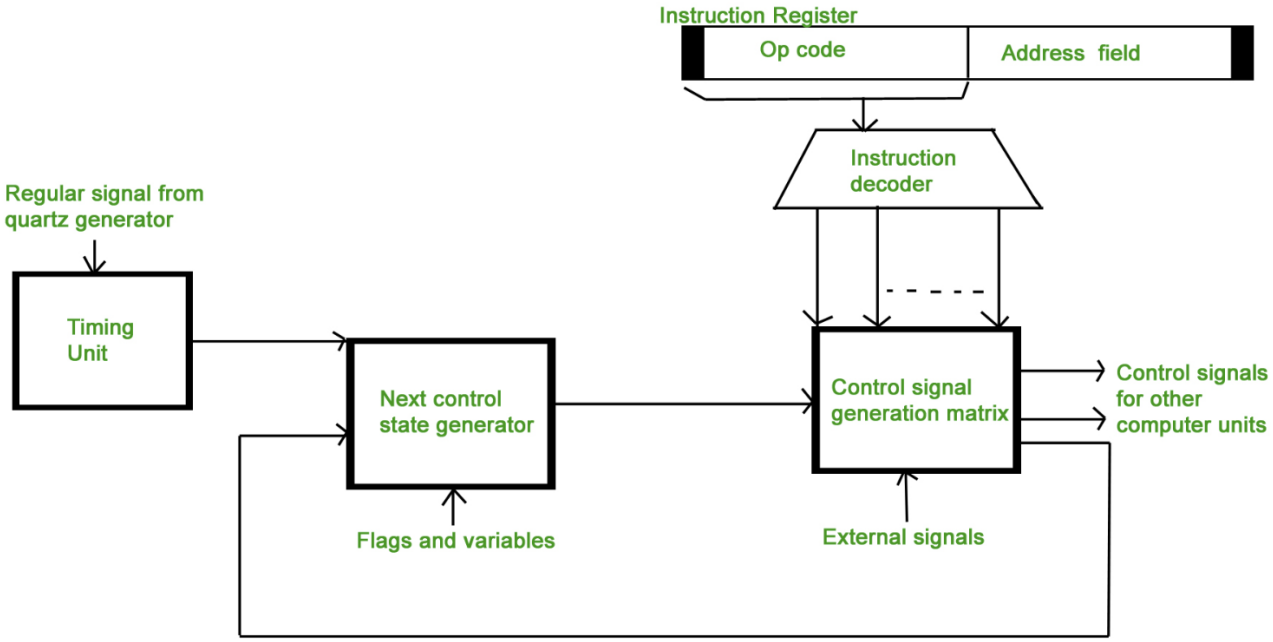
I segnali enA e enB vengono modificati in base alla microistruzione che sto eseguendo (me lo dice JCout)

CONSIDERAZIONI – TIPI DI CONTROLLO E CPU

- Un'unità di controllo **cablata** viene realizzata con logica combinatoria specificamente progettata per pilotare i diversi segnali di controllo nelle diverse «fasi» di elaborazione, mentre un'unità di controllo **microprogrammata** utilizza una memoria di controllo in cui sono memorizzate le microistruzioni che devono essere attivate in una specifica sequenza per fornire di volta in volta i segnali di controllo necessari.
- I due tipi di controllo sono impiegati per l'implementazione della CU all'interno della CPU per generare i segnali di controllo relativi alle diverse istruzioni dell'ISA.
  - Con il controllo cablato, ogni istruzione ha il suo circuito dedicato all'interno della CU (concettualmente semplice e veloce ma costoso e difficile da modificare ed estendere)
  - Con il controllo microprogrammato, per ogni istruzione ISA c'è un insieme di microistruzioni all'interno della micro ROM e la CU deve generare la sequenza corretta di segnali di controllo andando ad eseguire la sequenza corretta di microistruzioni (più complesso e lento per via degli accessi alla memoria, ma più flessibile)

CPU E CONTROLLO CABLATO

- L'hardware di controllo può essere realizzato come una macchina a stati che passa da uno stato all'altro ad ogni colpo di clock, in base al contenuto dell'instruction register, ai condition code e ai segnali esterni. Le architetture RISC sono tipicamente realizzate in logica cablata
  - Esempi: Intel 8085, Motorola 6802, Zilog 80



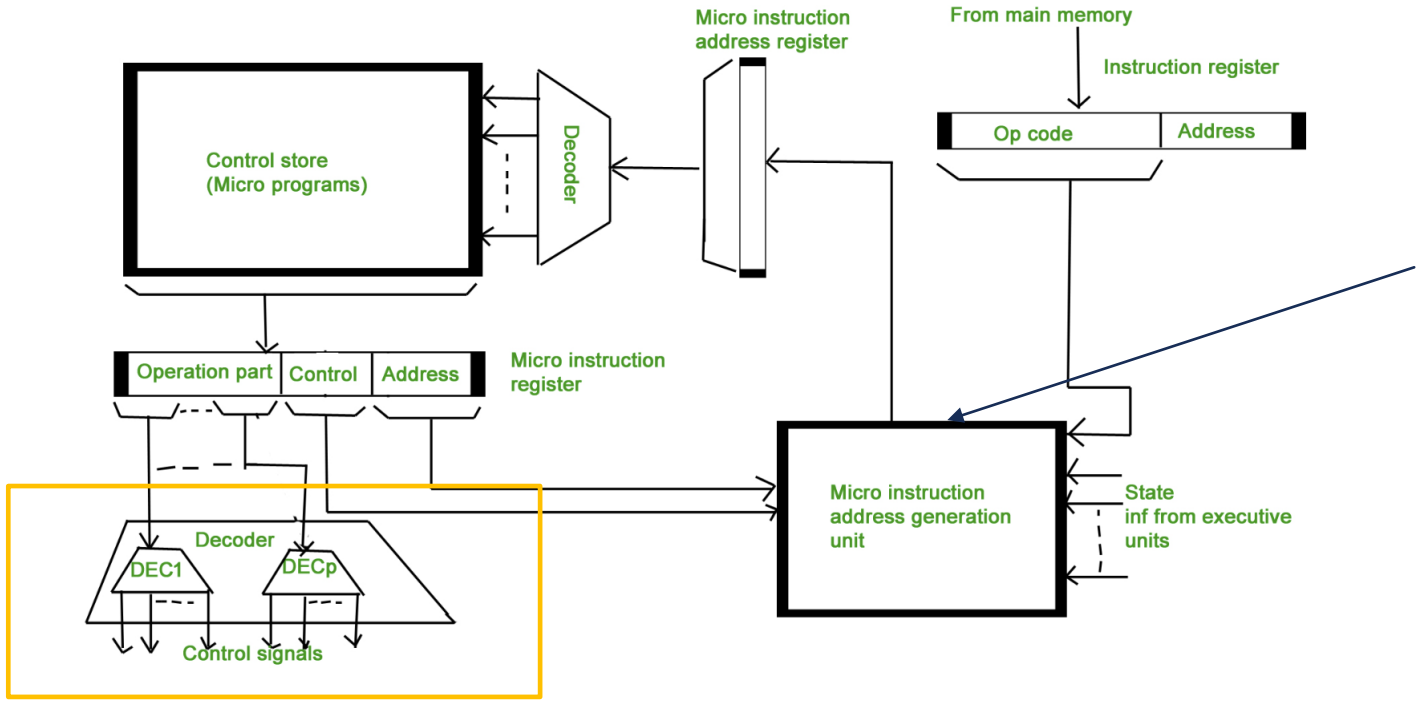
- Ogni segnale di controllo può essere ricavato come una funzione del codice operativo, dei modi di indirizzamento, dei condition code e della specifica fase di elaborazione in cui si trova la macchina
- A livello circuitale la FSM viene realizzata tramite un contatore (sequencer): le uscite del contatore codificano gli stati della macchina

Image credits: https://www.geeksforgeeks.org/computer-organization-hardwired-vs-micro-programmed-control-unit/

CPU E CONTROLLO MICROPROGRAMMATO

- I segnali di controllo associati alle diverse operazioni della CPU vengono memorizzati all'interno di **control words**
- Per ogni istruzione macchina c'è una sequenza di control words (microistruzioni) che viene detta micro-routine
- Tutte le micro-routines per tutte le istruzioni dell'ISA di una CPU sono memorizzate in una **Control Store**.
  - Esempi: Intel 8080, Motorola 68000

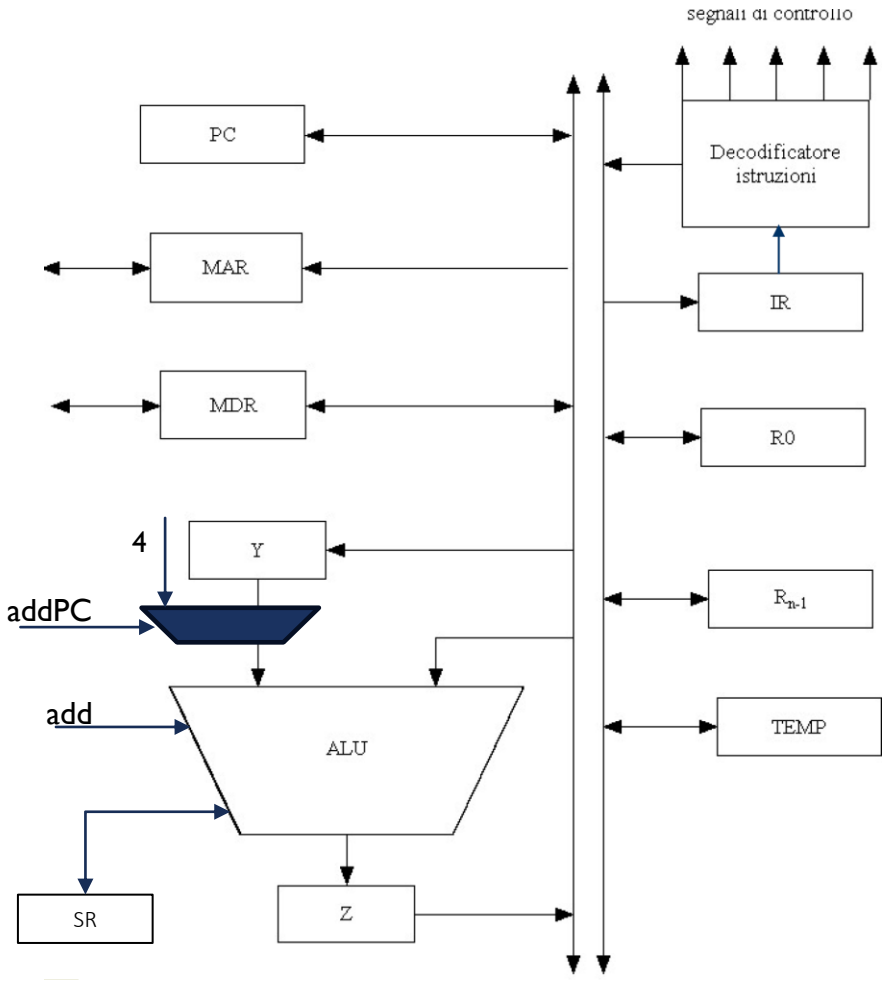
In base a come sono codificati i segnali di controllo si parla di unità microprogrammata **orizzontale** (1 bit nella control word per ogni segnale di controllo) o **verticale** (segnali di controllo codificati)



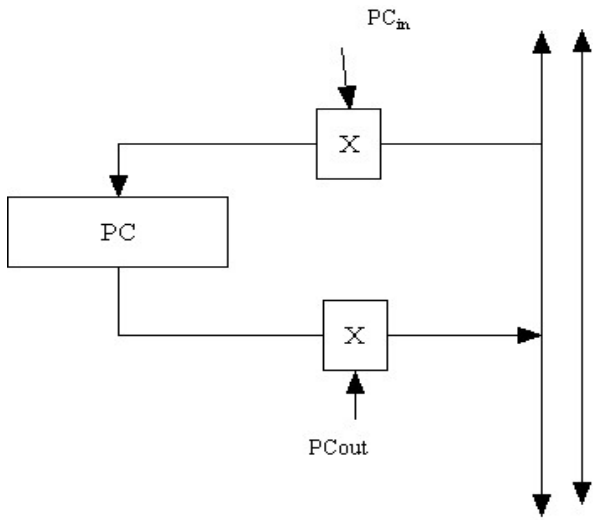
Viene solitamente indicato come sequencer

Image credits: https://www.geeksforgeeks.org/computer-organization-hardwired-vs-micro-programmed-control-unit/

CPU A CONTROLLO MICROPROGRAMMATO: ESEMPIO DI DATAPATH



- Ogni registro connesso al bus in modo bidirezionale deve avere due porte tristate pilotate da segnali di controllo indipendenti (Rin e Rout per ogni registro, non mostrato in figura)



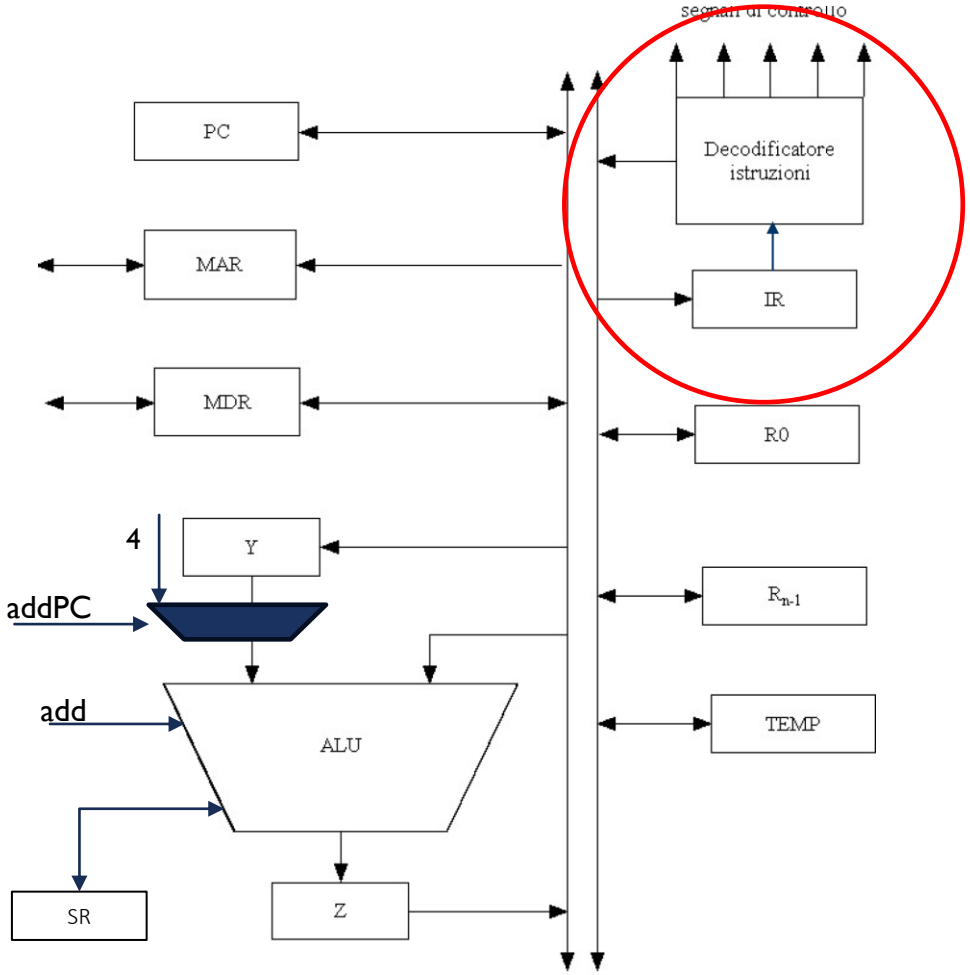
ESEMPIO DI ESECUZIONE DI UNA ISTRUZIONE: MICROFASI

- Si consideri la seguente istruzione (simil Motorola 68000): **Add (R3), R1**

- Essa richiede le seguenti azioni:

  1. Prelevare l'istruzione,
  2. Prelevare il primo operando (il contenuto della locazione puntata da R3),
  3. Eseguire l'addizione,
  4. Caricare il risultato nel registro R1.

ESEMPIO DI ESECUZIONE DI UNA ISTRUZIONE: MICROFASI



Add (R3), R1

1. Prelevare l'istruzione,
2. Prelevare il contento della locazione puntata da A3
3. Eseguire l'addizione,
4. Caricare il risultato nel registro D1

Assunzioni:

- Le istruzioni sono lunghe 4 byte
- Si può scrivere un solo registro alla volta sul bus

Microfasi:

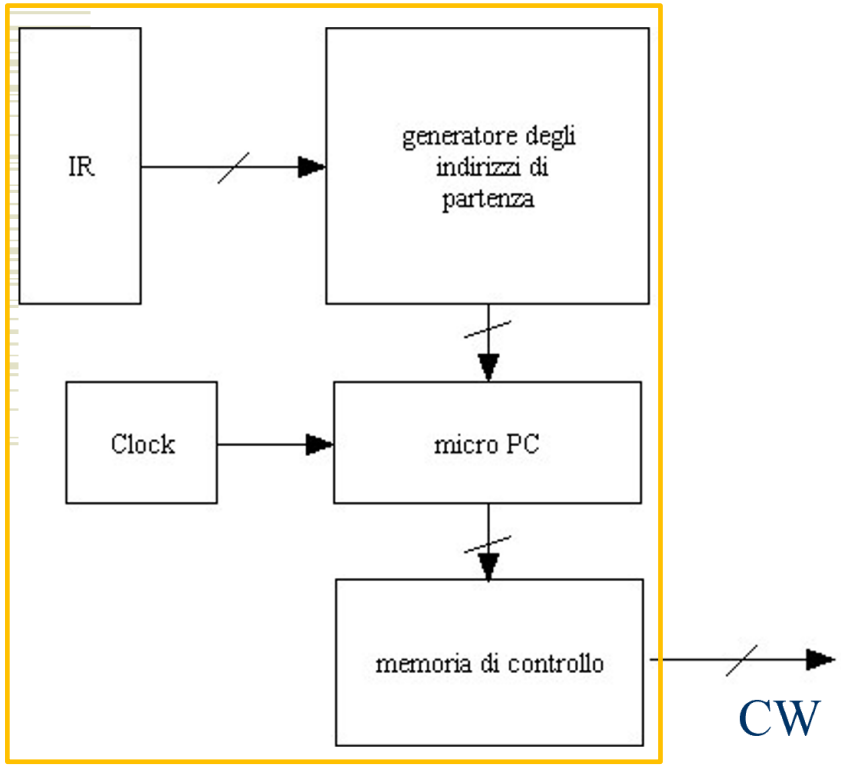
1: PCout, MARin, Read, Add; AddPc; Zin
2: Zout, Pcin, wait_for_mem_read
3: MDRin
4: MDRout, IRin
5: R3out, MARin, Read
6: R1out, Yin, wait_for_mem_read
7: MDRin
8: MDRout, Add, Zin
9: Zout, R1in ***goto step 1***

si attende che la mem asserisca un segnale che conferma l'avvenuta lettura)

fetch

CONTROLLO MICROPROGRAMMATO

- La CPU deve generare nella giusta sequenza i segnali di controllo per l'esecuzione delle istruzioni, secondo le microfasi precedentemente individuate
- Quando si adotta un controllo microprogrammato, la "sequenza di controllo" di una istruzione macchina è memorizzata all'interno di una memoria di controllo (**control store**) sotto forma di microistruzioni (**control word**)
- Un **decodificatore di istruzioni** si occupa di generare le control word servendosi di:
  - un microPC (contatore di microprogramma) tempificato dal clock che punta alle control word da fornire in output
  - un generatore di indirizzi di partenza per individuare l'inizio di ciascuna sequenza in base al codice operativo dell'istruzione macchina
  - un meccanismo per modificare il flusso di controllo in caso di salti



Decodificatore di istruzioni

TIPI DI CONTROLLO MICROPROGRAMMATO

- **Microprogrammazione ad organizzazione orizzontale**: si usa un bit per ogni segnale
  - La decodifica è molto semplice;
  - Le microistruzioni sono molto lunghe con pochi 1 attivi contemporaneamente;
- **Microprogrammazione ad organizzazione verticale**: si raggruppano i segnali mutuamente esclusivi
  - Si può usare uno schema di codifica binaria con semplici decodificatori: ad es. se abbiamo16 istruzioni della ALU bastano 4 bit, per gli n segnali di uscita dei registri dato uso log(n) bit, per gli n segnali di ingresso dei registri dato uso log(n) bit, ….
  - Il raggruppamento dei segnali richiede esplicitamente la presenza di hw aggiuntivo per i decodificatori che è però compensato dall'avere una memoria di controllo più piccola;
  - NOTA: è possibile raggruppare anche tutti i segnali insieme e non solo quelli mutuamente esclusivi in modo da avere CW di lunghezza minima; in questo caso però i circuiti di decodifica sono, ovviamente, più complessi

ORGANIZZAZIONE DELLA CONTROL STORE

- La control store deve contenere le micro-routine (e quindi le microistruzioni) per eseguire tutte le istruzioni dell'ISA di un processore, quindi le memorie di controllo sono di dimensioni notevoli
- Considerando tutte le possibili modalità di indirizzamento, in realtà, molte micro-procedure hanno diverse microistruzioni in comune

- Occorre organizzare la memoria in modo da individuare e scrivere una unica volte tutte le microistruzioni comuni a diverse operazioni. Esistono due tecniche maggiormente diffuse che aiutano a mantenere contenuta la dimensione di una control store:
  - **Condividere le parti comuni** di istruzioni diverse semplificando il modo in cui viene calcolato il valore da inserire nel microPC, ad esempio **scegliendo opportunamente gli indirizzi** delle microistruzioni in modo che la **distanza tra due microistruzioni che hanno parti in comune sia di un solo bit** (qualsiasi bit del microindirizzo) che possa facilmente essere valutato con una operazione di **bit OR-ing.**
  - **Includere** in ogni microistruzione un **campo indirizzo** che indica la locazione della microistruzione successiva (aumento la lunghezza della CW ma il sistema è più performante);

ESEMPIO DI BIT-ORING

- Consideriamo una istruzione di livello ISA che ammetta per un operando sia indirizzamento immediato sia indirizzamento indiretto: nella control store per la microroutine associata a questa istruzione ci saranno delle microistruzioni in comune per i due casi, e delle microistruzioni diverse.
- Assumendo di avere microistruzioni di 4 byte e la control store byte addressable, se la prima microistruzione diversa, quella relativa al modo di indirizzamento immediato, è memorizzata alla locazione x60 (%0110 0**0**00), allora conviene memorizzare la prima microistruzione relativa al modo di indirizzamento indiretto in una locazione vicina, che differisca di un solo 1 nell'indirizzo, per esempio x64 (%0110 0**1**00).
- In questo modo, la microroutine procede naturalmente fino alla microistruzione all'indirizzo x5C come se volesse eseguire quella a x60, se poi si verifica una condizione che specifica la modalità di indirizzamento indiretto, allora si utilizza una porta OR per cambiare il bit 3 dell'indirizzo.